

Optimizers and Training Best Practices

KAUST–Mawhiba IOAI Training

Week 2 · Day 8



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

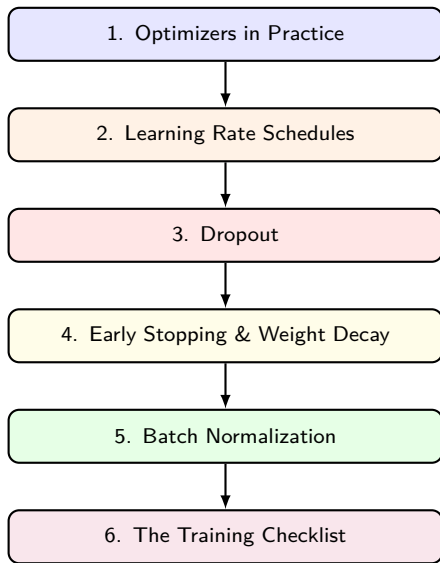
May 2, 2026

1. Recap: From Theory to Practice
2. Optimizers in Practice
3. Learning Rate Schedules
4. Dropout
5. Early Stopping
6. Weight Decay
7. Batch Normalization
8. The Training Checklist

Yesterday you learned *how neural networks learn*:

- ▶ **Forward pass** – compute predictions from inputs
- ▶ **Loss function** – measure how wrong we are
- ▶ **Backpropagation** – compute gradients via the chain rule
- ▶ **Gradient descent** – update weights: $w \leftarrow w - \alpha \nabla J$
- ▶ **Optimizers** – Momentum, AdaGrad, RMSProp, **Adam**, **AdamW**

Today's question: We know *how* networks learn.
But how do we make them learn *well*?



All optimizers update weights using gradients, but differ in *how*:

Optimizer	Key Idea
SGD	Vanilla gradient step
SGD + Momentum	Accumulate velocity to escape ravines
Adam	Adaptive LR per parameter + momentum
AdamW	Adam with decoupled weight decay

Rule of thumb: Start with **AdamW** — it is the most robust default.

SGD with Momentum

```
optimizer = torch.optim.SGD(model.parameters(),  
                             lr=0.01, momentum=0.9)
```

Adam

```
optimizer = torch.optim.Adam(model.parameters(),  
                              lr=1e-3)
```

AdamW (recommended default)

```
optimizer = torch.optim.AdamW(model.parameters(),  
                               lr=1e-3, weight_decay=0.01)
```

SGD + Momentum

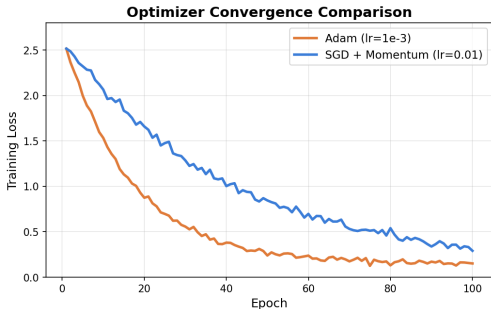
- ▶ Often **generalizes better**
- ▶ Needs careful LR tuning
- ▶ Slower to converge
- ▶ Popular in CV research

AdamW

- ▶ **Fast convergence**
- ▶ Less sensitive to LR choice
- ▶ Built-in weight decay
- ▶ Great default for most tasks

Practical tip: Prototype with AdamW ($1r=1e-3$). Once the model works, try SGD with momentum and a learning rate schedule for potentially better final accuracy.

Convergence: Adam vs SGD

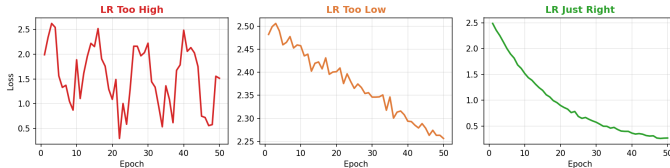


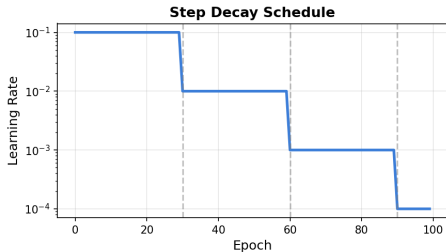
- ▶ **Adam:** Fast early progress, adaptive per-parameter LR
- ▶ **SGD + Momentum:** Slower start, but can reach a *flatter* minimum
- ▶ Flatter minima → better generalization

Why Schedule the Learning Rate?

- ▶ A **fixed learning rate** is rarely optimal throughout training:
 - **Too high:** Loss oscillates or diverges
 - **Too low:** Training is painfully slow
- ▶ **Solution:** Start with a high LR, then *decrease it* as training progresses.
- ▶ Intuition: take big steps early (explore), small steps later (fine-tune).

Effect of Learning Rate on Training



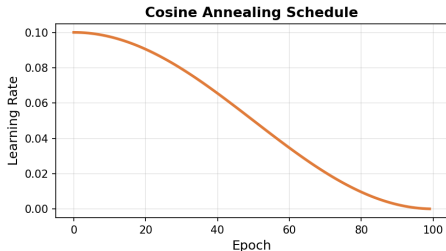


Idea: Multiply LR by a factor γ every N epochs.

$$\alpha_t = \alpha_0 \cdot \gamma^{\lfloor t/N \rfloor}$$

PyTorch

```
scheduler = StepLR(  
    optimizer,  
    step_size=30,  
    gamma=0.1)
```



Idea: Smoothly decrease LR following a cosine curve.

$$\alpha_t = \alpha_{min} + \frac{\alpha_0 - \alpha_{min}}{2} \left(1 + \cos \frac{\pi t}{T} \right)$$

PyTorch

```
scheduler = CosineAnnealingLR(  
    optimizer,  
    T_max=100)
```

Very popular in modern training — smooth decay, no hyperparameter for step size.

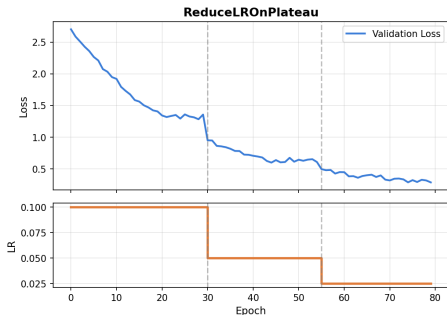
Idea: Only reduce LR when the loss *stops improving*.

PyTorch

```
scheduler = ReduceLROnPlateau(  
    optimizer,  
    patience=5,  
    factor=0.5)
```

```
# After each epoch:  
scheduler.step(val_loss)
```

Patience: how many epochs to wait before reducing.

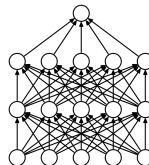


Which Schedule to Use?

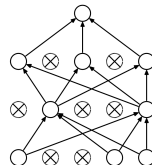
Schedule	Best For
Step Decay	Simple, works well for fixed-length training
Cosine Annealing	Modern default; smooth decay, easy to tune
ReduceLROnPlateau	When you don't know how many epochs you need

Practical tip: Use **Cosine Annealing** as your default. Switch to **ReduceLROnPlateau** if training length is unpredictable.

- ▶ **Dropout** randomly sets a fraction p of neuron outputs to zero during each forward pass.
- ▶ Forces the network to **not rely** on any single neuron.
- ▶ Acts as training an **implicit ensemble** of many sub-networks.



(a) Standard Neural Net

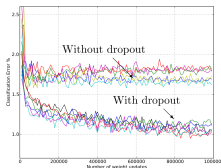


(b) After applying dropout.

Srivastava et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting”, JMLR 2014.

Why Does Dropping Neurons Help?

1. **Prevents co-adaptation:** Neurons cannot rely on specific partners → each neuron must learn *independently* useful features.
2. **Implicit ensemble:** Each forward pass uses a different random sub-network. At test time, the full network approximates an **average of all sub-networks**.
3. More independent features + diverse ensemble = **less overfitting**.

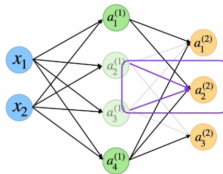


During Training:

- ▶ Randomly zero out neurons with probability p
- ▶ Scale remaining activations by $\frac{1}{1-p}$ (*inverted dropout*)

During Inference:

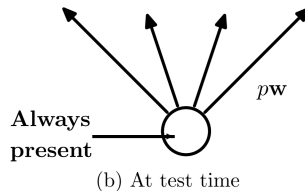
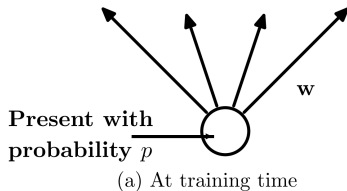
- ▶ Use **all** neurons (no dropout)
- ▶ No scaling needed (inverted dropout handled it)



Assume we have
present probability
 $p = 0.5$

During testing,
when nodes are
present again,
activations are
 $2 \times$ as large!

Key: PyTorch's `nn.Dropout` uses inverted dropout automatically. Just remember to call `model.train()` and `model.eval()`!



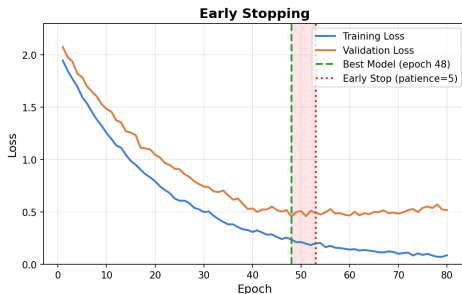
- ▶ **Without dropout:** Network memorizes training data (overfitting).
- ▶ **With dropout:** Test accuracy improves — the model generalizes better.

Adding Dropout to an MLP

```
class MLP(nn.Module):  
    def __init__(self):  
        super().__init__()  
        self.fc1 = nn.Linear(784, 256)  
        self.drop1 = nn.Dropout(p=0.5)  
        self.fc2 = nn.Linear(256, 10)  
  
    def forward(self, x):  
        x = F.relu(self.fc1(x))  
        x = self.drop1(x)    # applied after activation  
        return self.fc2(x)
```

Training vs Eval Mode

```
model.train()    # enables dropout  
model.eval()     # disables dropout
```



- ▶ Training loss keeps decreasing, but **validation loss starts increasing**.
- ▶ The gap = *overfitting*.
- ▶ **Early stopping**: Stop training when validation loss hasn't improved for patience epochs.
- ▶ Save the **best model** (lowest val loss).

Remember Day 4? Early stopping is another form of **regularization** — it limits model complexity by limiting training time.

PyTorch Pattern

```
best_loss = float('inf')
patience = 5
wait = 0

for epoch in range(max_epochs):
    train_one_epoch(model, train_loader)
    val_loss = evaluate(model, val_loader)

    if val_loss < best_loss:
        best_loss = val_loss
        wait = 0
        torch.save(model.state_dict(), 'best.pt')
    else:
        wait += 1
        if wait >= patience:
            print('Early stopping!')
            break
```

- ▶ Recall **L2 regularization** from Day 4: add $\lambda \sum w_j^2$ to the loss.
- ▶ In gradient descent, this changes the update rule:

$$w \leftarrow w - \alpha \nabla J - \alpha \lambda w = (1 - \alpha \lambda) w - \alpha \nabla J$$

- ▶ Each step **shrinks** weights by a factor $(1 - \alpha \lambda)$ — hence “weight decay.”

Problem with Adam: Adam normalizes gradients by their running variance. When L2 is part of the loss, the penalty gets normalized too, making regularization **inconsistent**.

AdamW (Loshchilov & Hutter, 2019) fixes this by applying weight decay *directly to the weights*, separate from the gradient:

Adam + L2

$$g_t \leftarrow \nabla J + \lambda w$$

$$m_t, v_t \leftarrow \text{update}(g_t)$$

$$w \leftarrow w - \alpha \frac{m_t}{\sqrt{v_t + \epsilon}}$$

Penalty is mixed into gradient $\rightarrow$ gets scaled by Adam's adaptive LR.

AdamW (Decoupled)

$$g_t \leftarrow \nabla J$$

$$m_t, v_t \leftarrow \text{update}(g_t)$$

$$w \leftarrow w - \alpha \frac{m_t}{\sqrt{v_t + \epsilon}} - \alpha \lambda w$$

Weight decay applied directly $\rightarrow$ consistent regularization.

PyTorch

```
optimizer = torch.optim.AdamW(model.parameters(),  
                                lr=1e-3, weight_decay=0.01)
```

- ▶ **Key insight:** Networks train better when inputs are normalized (zero mean, unit variance).
- ▶ So why not normalize *intermediate layers* too?
- ▶ **Problem:** Forcing zero-mean unit-variance everywhere may be too restrictive.
- ▶ **Solution:** Normalize first, then let the network *learn* the best scale and shift.

For each feature j in a mini-batch of size N :

1. **Normalize:**

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

where $\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j}$ and $\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2$

2. **Scale and shift** (learnable parameters):

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$

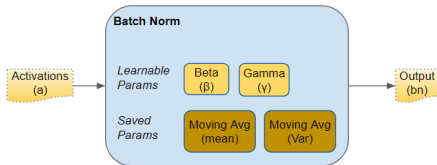
γ and β are **learned during training**, just like weights. The network decides how much normalization each layer actually needs.

After normalization, outputs are always zero-mean, unit-variance. But that might not be optimal for every layer!

- ▶ γ : “How much variance do I want?”
- ▶ β : “What should my mean activation be?”

Two extremes the network can learn:

- ▶ **Keep normalization:** $\gamma \approx 1$, $\beta \approx 0$
- ▶ **Undo normalization:** γ and β restore the original scale



Training

- ▶ Use statistics from the **current batch**
- ▶ Keep a **running average**:

$$\mu_{run} \leftarrow 0.9 \cdot \mu_{run} + 0.1 \cdot \mu_{batch}$$

Inference

- ▶ Use stored **running averages** ($\mu_{run}, \sigma_{run}^2$)
- ▶ No batch statistics needed!
- ▶ Fixed, deterministic

Common bug: Forgetting `model.eval()` at inference $\rightarrow$ BatchNorm uses batch stats instead of running stats $\rightarrow$ inconsistent predictions!

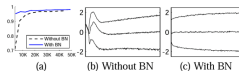


Figure 1: (a) The test accuracy of the MNIST network trained with and without Batch Normalization, vs. the number of training steps. Batch Normalization helps the network train faster and achieve higher accuracy. (b, c) The evolution of input distributions to a typical sigmoid, over the course of training, shown as {15, 50, 85}th percentiles. Batch Normalization makes the distribution more stable and reduces the internal covariate shift.

Advantages:

- ▶ Faster training
- ▶ Allows higher learning rates
- ▶ Reduces sensitivity to initialization
- ▶ Acts as mild regularization
- ▶ No overhead at test time

Watch out:

- ▶ Different behavior in train/eval mode
- ▶ Depends on batch size (small batches → noisy statistics)

For Fully Connected Layers

```
self.bn1 = nn.BatchNorm1d(256)  
x = F.relu(self.bn1(self.fc1(x)))
```

For Convolutional Layers (2D)

```
self.bn1 = nn.BatchNorm2d(16)    # 16 channels  
x = F.relu(self.bn1(self.conv1(x)))
```

Typical Ordering

Conv/Linear $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ Dropout

Remember: `model.train()` enables batch stats; `model.eval()` uses running stats.

1. **Start simple:** Train a basic model *without* regularization to get a baseline.
2. **Choose your optimizer:** AdamW (lr=1e-3, weight_decay=0.01) is a safe default.
3. **Add regularization:**
 - Dropout ($p = 0.3$ – 0.5 for FC layers)
 - Weight decay (already in AdamW)
 - Batch normalization (after every layer)
4. **Schedule the LR:** Cosine annealing or ReduceLROnPlateau.
5. **Monitor and stop:** Track training *and* validation loss. Use early stopping.
6. **Save the best model:** Always keep the checkpoint with the lowest val loss.

Summary: Tools in Your Toolbox

Technique	Purpose	Day Introduced
L1 / L2 Regularization	Constrain weight magnitude	Day 4
Cross-Validation	Robust evaluation	Day 4
Optimizer (AdamW)	Adaptive, fast training	Day 7
Learning Rate Schedule	Fine-grained convergence	Today
Dropout	Prevent co-adaptation	Today
Early Stopping	Prevent overtraining	Today
Weight Decay	Shrink weights smoothly	Today
Batch Normalization	Stabilize & speed up training	Today

Next up (Day 9): Convolutional Neural Networks — applying everything we've learned to *images*.

Credits

KAUST Academy

King Abdullah University of Science and Technology

Batch normalization and dropout content adapted from KAUST Academy deep learning materials

KAUST-Mawhiba IOAI 2026 Training Program